

This is a **much stronger system than the installer**, architecturally. You've clearly internalized LFS constraints and built a disciplined orchestration layer. That said, there are still several **failure-mode and shell semantics issues** that will bite you under stress (interruptions, partial builds, edge filesystem states).

I'll go through your requested areas, then give a **systems-level critique**.

High-Level Assessment

What you got right

- Proper use of `set -euo pipefail` in orchestrator
- Phase-based resumability is clean and intuitive
- `DESTDIR` + tar pipeline is **correct in principle**
- Chroot environment isolation is strong (`env -i`)
- Logging strategy is consistent and usable
- Thoughtful LFS alignment (this is not “random bash”)

Where this will break

- Signal handling is **not actually cleaning up state**
- Mount lifecycle is **not transactional**
- Tar-based deployment has **edge-case footguns**
- Checkpointing is **not crash-safe**
- `set -e` behavior is **inconsistent across subshells/pipes**

1. Shell Scripting Safety ⚠️ (Moderate Risk)

☑️ Good

- You consistently quote variables ("\${IGOS}", "\${BUILD_USER}")
- set -euo pipefail is present (critical)

✗ Problem: set -e + pipelines + tee

Example:

```
su - "${BUILD_USER}" -c "..." 2>&1 | tee -a "${BUILD_LOG}"
```

With pipefail, this is **mostly correct**, but:

- tee can succeed even if upstream fails under certain conditions
- You are not checking \${PIPESTATUS[0]}

👉 Result: **false positives on success**

Fix

Wrap pipelines:

```
set -o pipefail
su - "${BUILD_USER}" -c "..." 2>&1 | tee -a "${BUILD_LOG}"
rc=${PIPESTATUS[0]}
if [ $rc -ne 0 ]; then
    return $rc
fi
```

✗ Problem: unquoted globs

```
cp -n "${SOURCES}"/* "$IGOS/sources/"
```

If empty → expands to literal * → failure or weird behavior.

Fix

```
shopt -s nullglob
files=("${SOURCES}"/*)
if [ $#files[@] -gt 0 ]; then
  cp -n "${files[@]}" "$IGOS/sources/"
fi
```

2. Chroot Mount/Unmount Correctness (High Risk)

✗ Problem: teardown is not exhaustive

```
umount $IGOS/dev
```

Issues:

- No recursive unmount
- No lazy fallback
- No mount tracking

✗ Problem: setup does not track mounts

You rely on:

```
if ! mountpoint -q ...
```

But:

- No record of what THIS run mounted
- Cannot safely unwind partial state

✗ Problem: interruption = leaked mounts

Your trap:

```
trap cleanup SIGINT SIGTERM
```

But cleanup():

```
exit 130
```

→ **Does NOT call chroot-teardown**

Fix (critical)

In orchestrator:

```
cleanup() {  
    log "Cleaning up mounts..."  
    bash "${SCRIPTS}/chroot-teardown.sh" || true  
    exit 130  
}
```

Stronger fix (recommended)

Use:

```
umount -R "$IGOS" 2>/dev/null || umount -l "$IGOS"
```

3. DESTDIR Staging & Deployment

(Moderate–High Risk)

Your method:

```
tar -C staging -cf - . | tar -C / -xf - --keep-directory-symlink
```

Good

- Preserves metadata
- Avoids cp pitfalls
- Slackware-style → proven approach

Edge Case: symlink traversal

Even with:

```
--keep-directory-symlink
```

You still risk:

- overwriting through symlinks
- especially /usr/lib -> /lib scenarios depending on system state

✗ Missing safety: path sanitization

You do NOT check for:

- ../ paths
- absolute paths inside tar (unlikely but possible)

✗ Missing dry-run validation

No:

`tar -tf`

or preflight check.

Recommendation

Add:

```
tar -C "$dest" -cf - . | tar -tf - > /dev/null
```

AND:

```
tar -C "$dest" -cf - . \  
| tar -C / -xf - \  
  --no-overwrite-dir \  
  --keep-directory-symlink \  
  --numeric-owner
```

Bigger concern

You are effectively doing a **live system mutation without rollback**.

If interrupted:

- half-installed package
- inconsistent filesystem

4. Checkpoint Save/Restore (Moderate Risk)

Problem: not atomic

```
tar -C "$IGOS" -czf "$checkpoint" .
```

If interrupted:

- corrupt checkpoint
- no validation

Problem: teardown not guaranteed

```
chroot-teardown.sh || true
```

→ you may snapshot **mounted state**

Problem: no integrity verification

No:

```
tar -tzf
```

Recommendations

- Write to temp file first:

```
checkpoint_tmp="${checkpoint}.tmp"  
tar ... -czf "$checkpoint_tmp"  
mv "$checkpoint_tmp" "$checkpoint"
```

- Verify:

```
tar -tzf "$checkpoint" > /dev/null
```

5. CWD Reset Between Phases ☒ (Good, but incomplete)

You added:

```
cd "$workdir"
```

This is correct.

✗ Edge case: subshell leakage

If a build script does:

```
(cd subdir && make)
```


→ fine

But if it does:

cd subdir

→ persists

You handle this correctly per phase.

Remaining issue

Environment leakage:

unset -f configure build check do_install post_install

Good — but:

- variables can still leak
- exported env persists

Recommendation

Wrap builds in subshell:

```
(  
  source "$build_script"  
  ...  
)
```

6. Package Tracking Correctness

(Moderate Risk)

Strong design

- Slackware-style manifests
- deterministic file list
- archive generation clean

Problem: race conditions (minor)

```
install_log=$(ls -t ... | head -1)
```

If multiple builds:

→ nondeterministic

Problem: file counting

```
find "$PKG_DEST" -not -type d | wc -l
```

Counts:

- symlinks
- device nodes (if any)

Probably fine, but not precise.

✗ Problem: pkg_remove is dangerous

```
rm -f "$fullpath"
```

No protection against:

- removing shared files
- removing critical dirs if manifest is wrong

Recommendation

- Add whitelist protections (/usr, /etc, etc.)
- Or require confirmation for removal

7. Signal Handling 🚨 (Critical)

✗ Current behavior

```
trap cleanup SIGINT SIGTERM
```

But:

```
cleanup() {  
    exit 130  
}
```

🚨 Problem

On Ctrl+C:

- mounts remain
- partial filesystem state
- possible corruption

Required fix

```
cleanup() {  
    log "Interrupt received. Cleaning up..."  
  
    if mountpoint -q "$IGOS/dev"; then  
        bash "${SCRIPTS}/chroot-teardown.sh" || true  
    fi  
  
    log "Cleanup complete."  
    exit 130  
}
```

Systemic Risks (Most Important Section)

1. Non-atomic build model

Your system is:

“mutable root filesystem with progressive writes”

This means:

- failure mid-package = undefined state

2. No journaling / state tracking

You track phases, but not:

- installed packages (during build)
- partial installs

3. No isolation boundary

Everything installs directly into:

/mnt/igos

Better model:

- overlayfs
- or snapshot-based (btrfs/zfs)

4. Trust model is absolute

You assume:

- all build.sh are correct
- no malicious or broken scripts

This is fine for LFS, but worth noting.

Priority Fix List

Must fix before heavy use

1. Fix signal cleanup (mount teardown)
2. Fix pipeline exit handling (PIPESTATUS)
3. Make checkpointing atomic
4. Add mount fallback (umount -R / lazy)

Should fix soon

5. Remove glob hazards
6. Add tar validation before deploy
7. Add environment isolation for builds (subshell)

Nice-to-have (but powerful)

8. Add build journaling (package-level checkpoint)
9. Add dry-run mode
10. Add filesystem snapshot support

Final Verdict

This is **competent, production-adjacent orchestration**, not a toy.

Compared to your installer:

- Installer = **architecturally clean but unsafe**
- Orchestrator = **operationally viable but missing resilience**

If you fix:

- signal cleanup
- pipeline correctness
- checkpoint atomicity

→ this becomes a **serious LFS automation system**.

If you want, I can:

- Design a **crash-safe checkpoint + resume model**
- Or refactor pkg_deploy into a **transactional install (staging + commit phase)**